Algorithms

Midterm Review

계산량을 세는 법부터 최적 선택을 증명하는 법까지 한 흐름으로 복습한다.

Lecture 2–11과 Practice, Assignment 1–2를 연결한다.
핵심은 입력·상태·활성 구간·적용 조건을 말로 설명하는 능력이다.

Review Map

각 단원은 서로 다른 질문을 풀지만, 모두 정확한 상태와 불변식을 요구한다.

Analysis·Recurrence: 무엇을 세고 어떻게 합산하는가.

Sorting·Selection: 어느 구간과 순위가 확정되는가.

DP·Greedy: 선택을 저장할지, 즉시 확정할지 어떻게 증명하는가.

Practice: Value and Cost

계산 결과와 실행 횟수는 다른 값이다. 먼저 세는 단위를 고정한다.

Stage	Body runs	Result	Cost
j=1	1	sum=1	2
j=2	2	sum=3	4
j=3	3	sum=6	6
Array loop	3	A=〔0,1,2〕	3
Total	9	sum=6	15

n=3, s_0=0에서 삼각 반복은 sum=6을 만들지만 연산 비용은 15다.
뒤의 배열 반복은 중첩 반복과 곱하지 않고 더한다.

Asymptotic Bounds

Worst case와 Big-O는 같은 축이 아니다. Θ는 상한과 하한을 함께 맞춘다.

ASYMPTOTIC NOTATIONS

- Lower bound와 Upper bound는 Tight하게 잡아야 함
- Tight하지 않은 예시

	Algorithm 1	Algorithm 2	Algorithm 3
T(n)	$2n+1$	$2n^2+1$	2
점근적 분석	n	n^2	1
Big-O	$O(n), O(n^2), \ldots$	$O(n^2), O(n^3), \ldots$	$O(1), O(n), \ldots$
Big-Omega	$\Omega(n), \Omega(1)$	$\Omega(n^2), \Omega(n), \ldots$	$\Omega(1)$
Big-Theta	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$

국민대학교 KOOKMIN UNIVERSITY

27

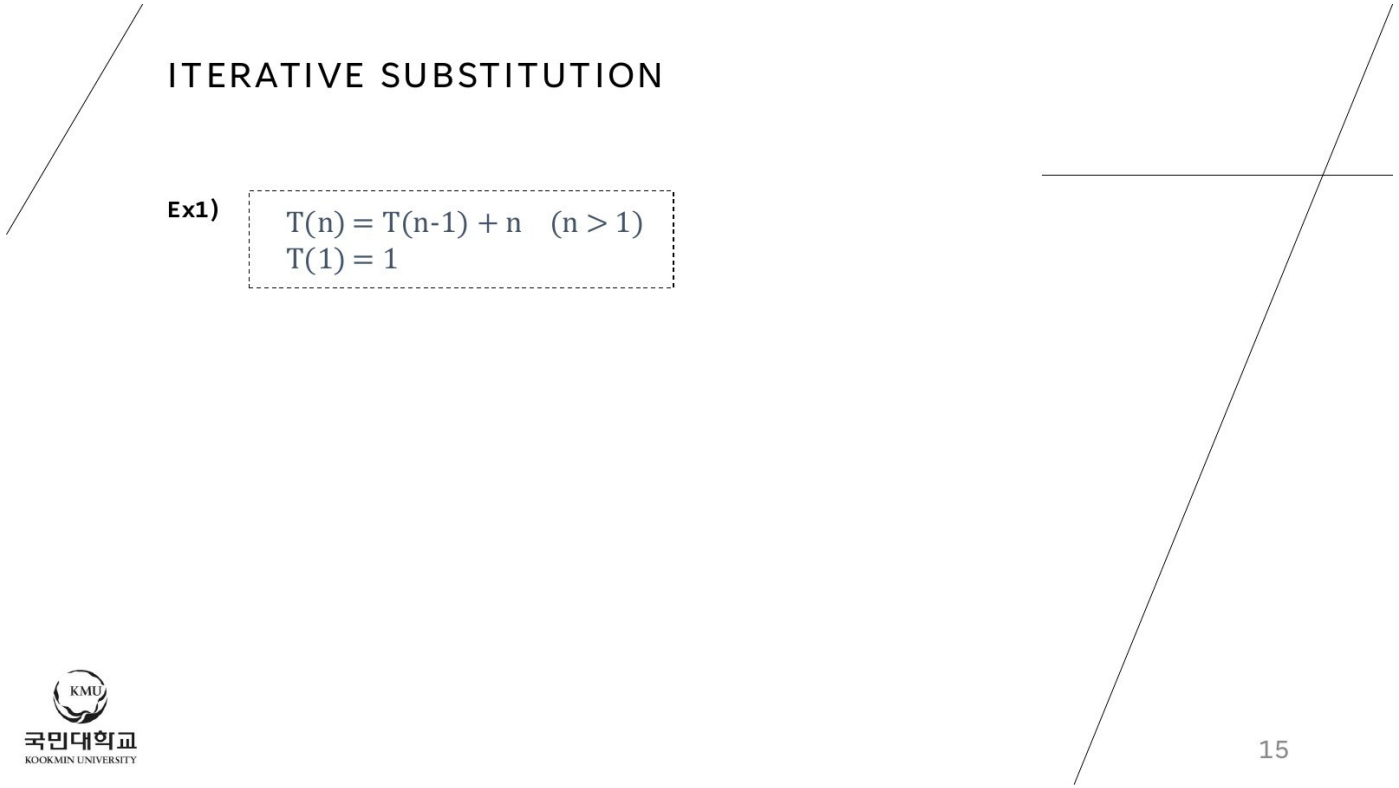
정확한 횟수, 입력 case, 점근 bound를 분리한다.

증명에는 상수 c와 시작점 n_0가 필요하다.

Lecture 2, p. 27: O, Ω, Θ와 tight bound의 차이

Recurrence by Substitution

재귀 호출 수와 현재 호출이 수행하는 일을 분리해야 점화식이 맞는다.



T(n)=T(n−1)+n은 n+(n−1)+…+1로 펼쳐진다.

반복 대입으로 형태를 찾고,
귀납으로 bound를 확인한다.

국민대학교 KOOKMIN UNIVERSITY

Lecture 3, p. 15: iterative substitution의 시작 식

Master Theorem Cases

f(n)을 임계 함수 n^(log_b a)와 비교하고 다항식 차이까지 확인한다.

Case	Comparison	Result	Assignment 1
1	f polynomially smaller	Θ (n^log_b a)	(c)
2	same order	Θ (n^log_b a log n)	(d) (e) (f)
3	f polynomially larger	Θ (f)	(a) (b) (g)

Case 3은 a·f(n/b) ≤ c·f(n), c < 1인 정규성 조건도 필요하다.
Assignment 1의 7문제는 세 case로 묶어 복습한다.

Sorting Comparison I

같은 $\Theta(n^2)$ 정렬도 입력 순서, 안정성, 이동 방식이 다르다.

Algorithm	Best	Worst	Stable
Bubble	$\Theta(n)$*	$\Theta(n^2)$	Yes*
Selection	$\Theta(n^2)$	$\Theta(n^2)$	No
Insertion	$\Theta(n)$	$\Theta(n^2)$	Yes*
Merge	$\Theta(n \log n)$	$\Theta(n \log n)$	Yes*

비교 횟수와 이동 횟수를 따로 본다.
Stable은 동점 순서, in-place는 보조 공간에 관한 성질이다.

Sorting Comparison II

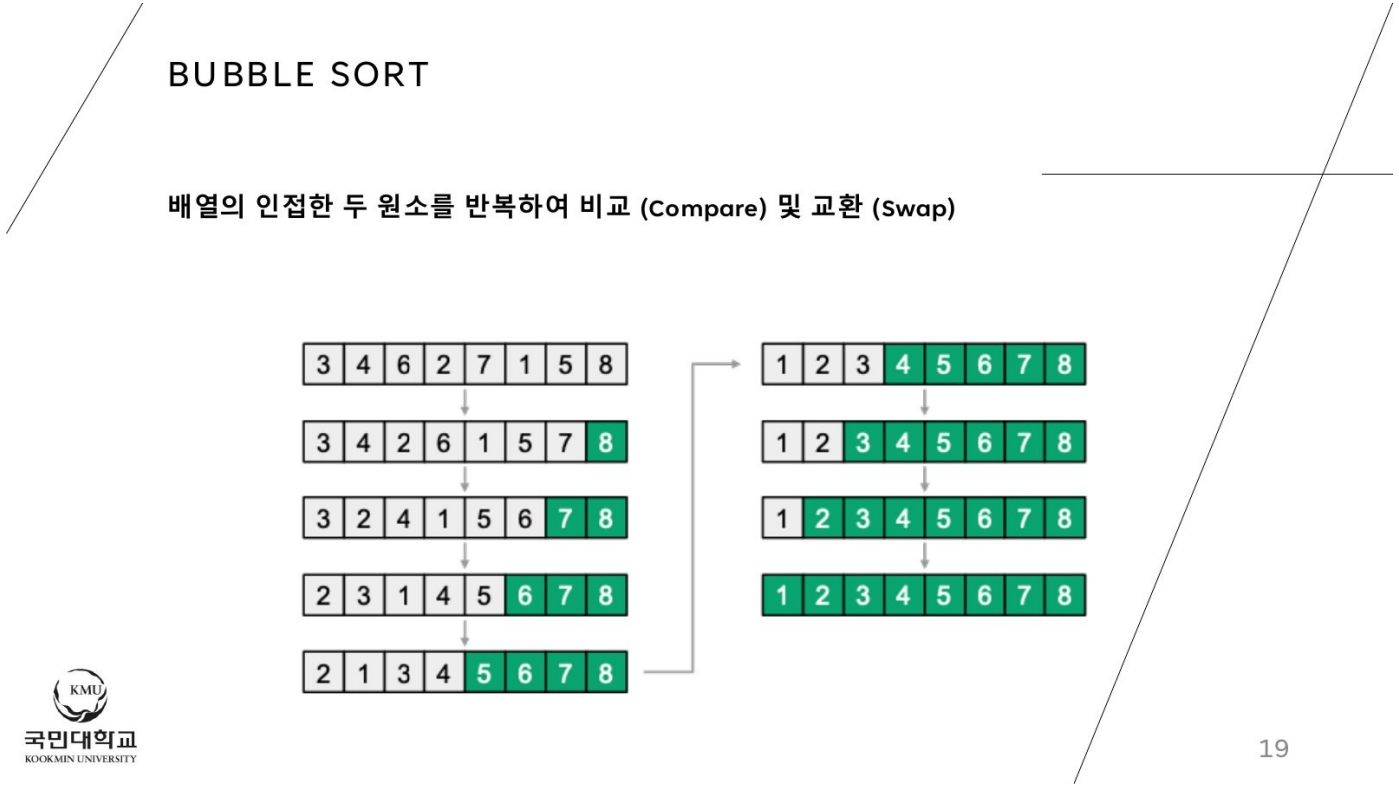
시간복잡도만 외우면 적용 조건과 메모리 비용을 놓친다.

Algorithm	Time (Θ)	Space	Key condition
Quick	E n log n / W n^2	stack	balanced pivot
Heap	W n log n	Θ(1)	heap invariant
Counting	n+r	Θ(n+r)	bounded keys
LSD Radix	k(n+r)	digit sort	stable digit sort

E는 기대, W는 최악 시간이다. Quick은 pivot 품질, Heap은 heap 불변식이 핵심이다.
Counting과 Radix는 키 범위 r과 자릿수 k를 비용에 포함한다.

Bubble Sort Trace

한 pass가 끝날 때마다 미정렬 구간의 최댓값 하나가 오른쪽에 확정된다.



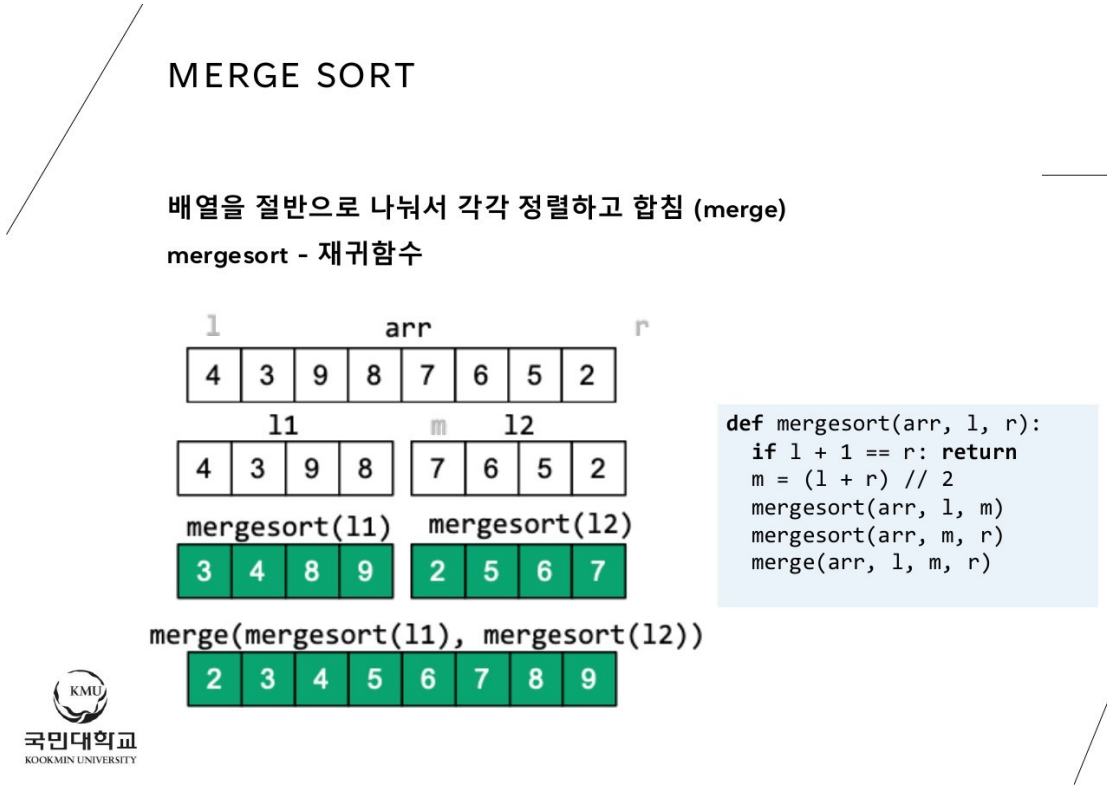
현재 배열과 확정 suffix를 함께 표시한다.

조기 종료가 없으면 이미 정렬된 입력도 $\Theta(n^2)$다.

Lecture 4, p. 19: pass마다 늘어나는 확정 suffix

Merge Sort Trace

정렬된 두 하위 구간을 합칠 때만 상위 구간 전체가 정렬된다.



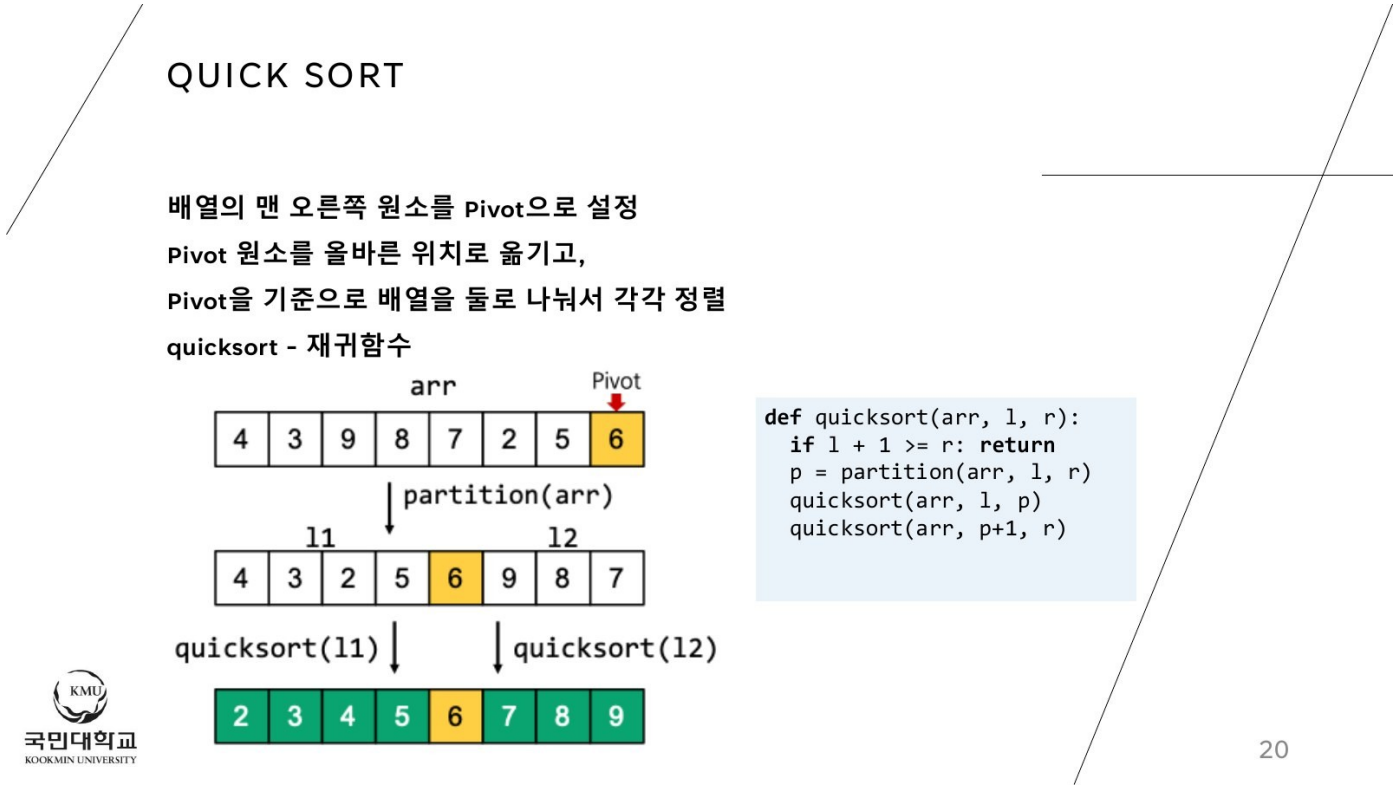
각 merge는 구간 길이에 선형이다.

log n개 레벨마다 총 n개 원소를 처리해 $\Theta(n \log n)$이 된다.

Lecture 5, p. 12: 분할한 두 배열의 정렬과 merge

Quick Sort Trace

Partition 직후 확정되는 것은 pivot 위치 하나다.



왼쪽과 오른쪽 구간의 내부 순서는 아직 정렬되지 않았다.

불균형 pivot이 반복되면 $T(n)=T(n-1)+\Theta(n)$이 된다.

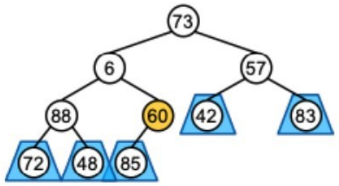
Lecture 5, p. 20: pivot 6의 partition과 두 재귀 구간

Build-Heap

마지막 내부 노드부터 sift-down하면 자식 subtree가 이미 heap이라는 조건을 쓸 수 있다.

BUILD-HEAP

- (자식을 가진) 부모 노드 맨 뒤에서부터 Sift down 연산을 반복 적용




```
def build_heap(arr):
  n = len(arr)
  for i in range(n//2-1,-1,-1):
    siftdown(i, arr, n)
```

국민대학교 KOOKMIN UNIVERSITY

13

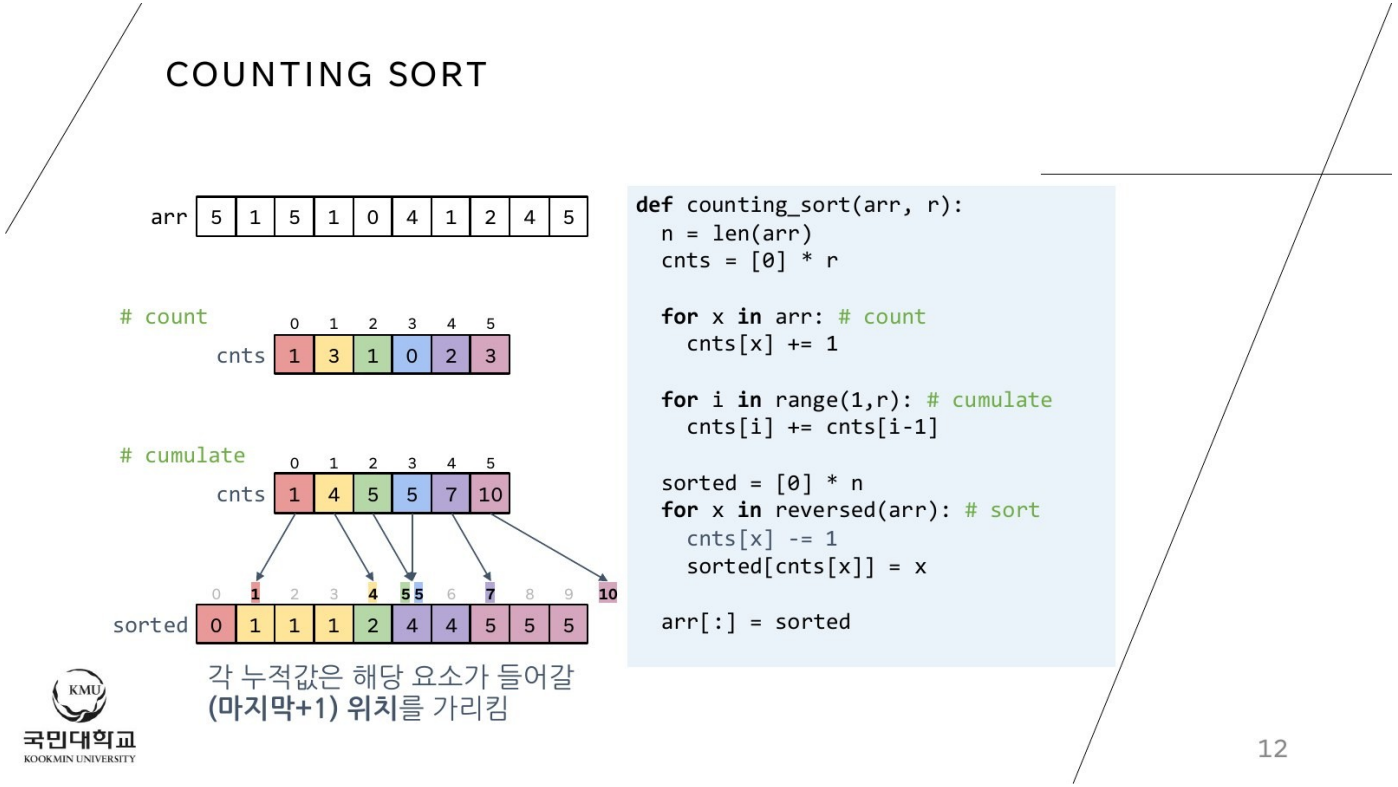
각 노드 비용은 높이에 비례하지만 높은 노드는 적다.

따라서 bottom-up build-heap은 $\Theta(n)$이다.

Lecture 6, p. 13: 마지막 내부 노드에서 시작하는 build-heap

Counting Sort

누적 개수는 각 키가 출력에서 끝나는 다음 위치를 알려 준다.




```
def counting_sort(arr, r):
  n = len(arr)
  cnts = [0] * r

  for x in arr: # count
    cnts[x] += 1

  for i in range(1,r): # cumulate
    cnts[i] += cnts[i-1]

  sorted = [0] * n
  for x in reversed(arr): # sort
    cnts[x] -= 1
    sorted[cnts[x]] = x

  arr[:] = sorted
```

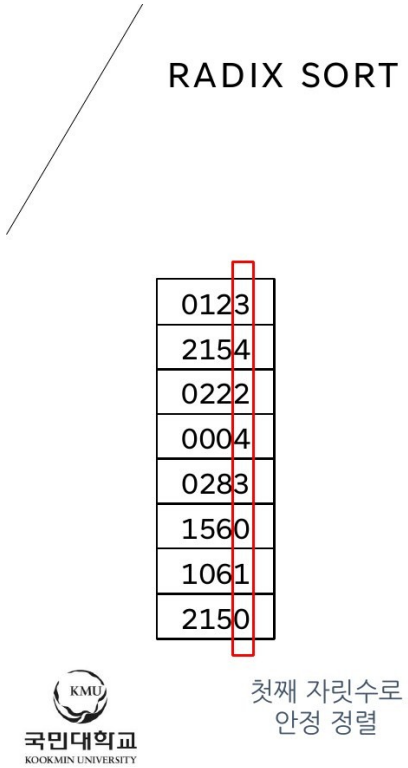
입력을 뒤에서 읽으면 같은 키의 상대 순서가 보존된다.

키는 $0 \le key < r$ 범위여야 한다.

Lecture 7, p. 12: count, cumulative count, stable placement

LSD Radix Sort

낮은 자리부터 정렬할 때 이전 자리의 순서를 보존해야 한다.



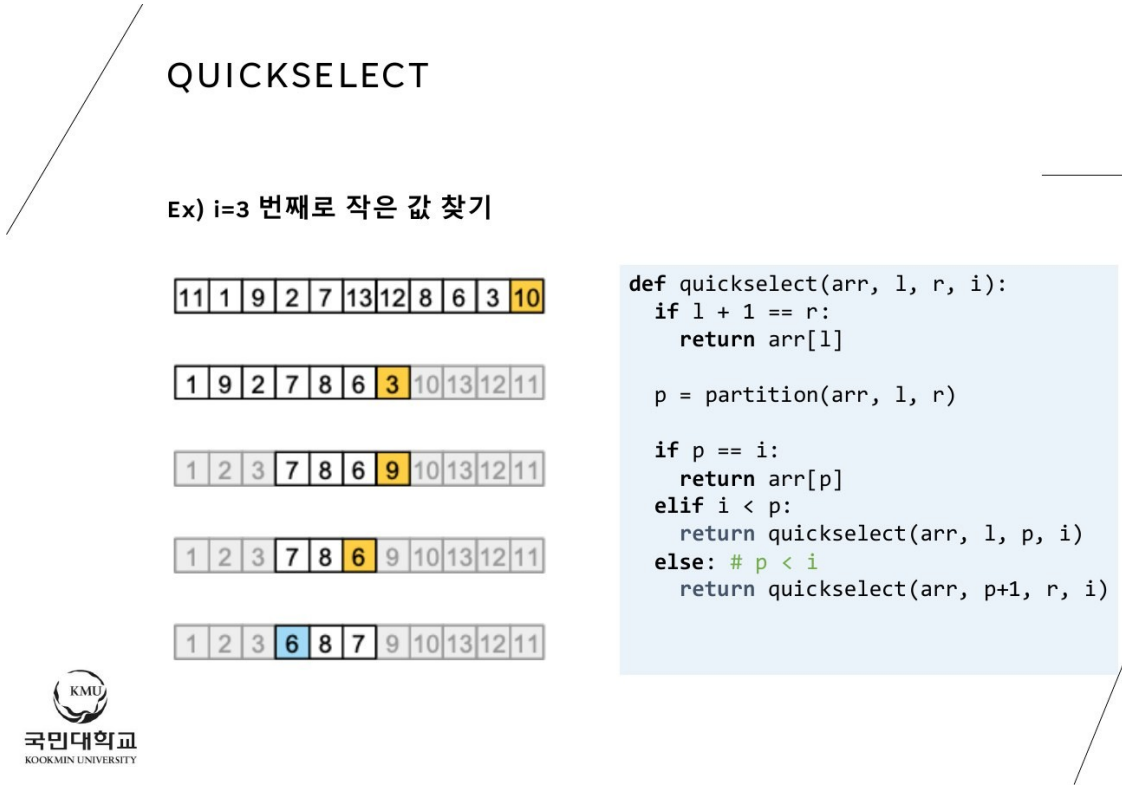
각 digit sort가 stable이면 높은 자리가 같을 때 낮은 자리 순서가 유지된다.

비용은 k개 자리마다 $\Theta(n+r)$다.

Lecture 7, p. 21: 첫 번째 LSD 자리 선택

Quickselect

Pivot의 rank와 목표 rank를 비교해 필요한 한쪽만 계속 탐색한다.



Lecture 8, p. 13: 목표 rank를 포함한 구간만 남기는 과정

전체 정렬이 필요 없어 기대 시간은 $\Theta(n)$이다.

극단 pivot이 반복되는 경로의 최악 시간은 $\Theta(n^2)$다.

Median of Medians

5개 그룹의 중앙값을 다시 선택하면 극단적으로 나쁜 pivot을 막을 수 있다.

MEDIAN OF MEDIANS

1. 입력으로 들어온 배열 원소들을 5개씩 그룹으로 묶음
2. 각 그룹마다 중앙값을 구함
3. 모든 중앙값들의 중앙값을 Pivot으로 사용하여 Partition 함수 실행

⇒ Pivot이 반드시 배열 내에서 30%~70% 위치에 있음을 보장해줌



국민대학교 KOOKMIN UNIVERSITY

26

각 방향에 약 3n/10개 원소가 보장된다.

$T(n) \leq T(n/5)+T(7n/10)+O(n)$ 이므로 최악 $O(n)$이다.

Lecture 8, p. 26: pivot 15 양쪽에 보장되는 원소 집합

DP State Construction

점화식은 외우는 식이 아니라 가능한 마지막 선택을 빠짐없이 나눈 결과다.

Stage	Question	Failure signal
State + Choice	무엇과 어떤 선택인가	정보 부족·중복
Recurrence + Base	작은 상태와 빈 문제는	누락·잘못된 연산
Order + Answer	참조가 먼저 계산됐나	미계산 참조
Sentinel + Restore	불가능·경로를 어떻게 표시하나	0과 미계산 혼동

상태가 다음 결정을 내리기에 충분한 정보를 담아야 한다.
값과 실제 선택을 복원하는 정보는 따로 저장할 수 있다.

Matrix Path

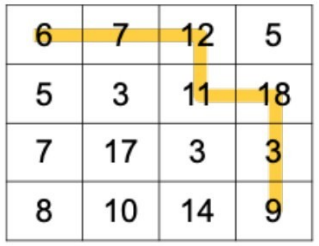
현재 칸의 마지막 이동은 위 또는 왼쪽뿐이므로 두 이전 상태 중 큰 값을 고른다.

MATRIX PATH PROBLEM

- **Bottom-up 방식 (Tabulation) 코드**

```
def matrix_path(mat):
  n = len(mat)
  dp = [[0] * n+1 for _ in range(n+1)]

  for i in range(n):
    for j in range(n):
      dp[i][j] = max(dp[i][j-1], dp[i-1][j]) + mat[i][j]
```



국민대학교 KOOKMIN UNIVERSITY

19

$P[i,j]=M[i,j]+\max(P[i-1,j],P[i,j-1])$.

강조 경로 합은 66이지만 전체 행렬의 최적값은 68이다.

Lecture 9, p. 19: bottom-up 순서와 강조 경로

Matrix-Chain Order

행렬 순서는 유지하고 괄호 위치만 바꿔 스칼라 곱셈 수를 최소화한다.

MATRIX MULTIPLICATION ORDER PROBLEM

- **재귀 특성 파악 (점화식 세우기)**

$S(i, j)$ 을 $A_iA_{i+1}...A_{j-1}$ 의 최소 연산량이라 하면

$$S(s, e) = \min(S(s, i) + S(i, e) + p_s p_i p_e \text{ for } s < i < e)$$

Ex) $A_1A_2A_3A_4A_5A_6A_7$ 문제의 경우

$(A_1)(A_2A_3A_4A_5A_6A_7)$
$(A_1A_2)(A_3A_4A_5A_6A_7)$
$(A_1A_2A_3)(A_4A_5A_6A_7)$
$(A_1A_2A_3A_4)(A_5A_6A_7)$
$(A_1A_2A_3A_4A_5)(A_6A_7)$
$(A_1A_2A_3A_4A_5A_6)(A_7)$

이들 중 최소 연산량을 보이는 것이 정답!

국민대학교 KOOKMIN UNIVERSITY

24

Lecture 9, p. 24: 구간을 두 부분으로 나누는 점화식

상태 C〔s,e〕는 반열린 구간 〔s,e) 의 최소 비용이다.

모든 분할점 k를 비교하고 최솟값과 k를 함께 저장한다.

Longest Common Subsequence

마지막 문자가 같으면 함께 쓰고, 다르면 한쪽 마지막 문자를 버리는 두 경우를 비교한다.

LONGEST COMMON SUB-SEQUENCE PROBLEM

Q1) 최적 부분 구조를 갖는가? (= 문제를 재귀적으로 표현할 수 있는가?)

- (case 1) X와 Y의 마지막 문자가 **같을 때**

  LCS(X,Y) = LCS(X[:-1],Y[:-1]) + 1

- (case 2) X와 Y의 마지막 문자가 **다를 때**

  LCS(X,Y) = max(LCS(X[:-1],Y), LCS(X,Y[:-1]))

국민대학교 KOOKMIN UNIVERSITY

10

빈 접두사 행과 열은 0으로 시작한다.

길이 표를 역추적하면 실제 LCS를 복원할 수 있다.

Lecture 10, p. 10: 마지막 문자 일치 여부에 따른 두 recurrence

Making It 1

최솟값 테이블과 이전 상태 테이블을 함께 저장해야 연산 수와 경로를 모두 얻는다.

MAKING IT 1 PROBLEM (2)

- **Bottom-up 방식 (Tabulation) 으로 풀 경우**

값 테이블

1	2	3	4	5	6	7	8
0	1	1	2	3	2	3	3

경로 저장용 테이블

1	2	3	4	5	6	7	8
0	1	1	2	4	3	6	4

국민대학교 KOOKMIN UNIVERSITY

27

D[i]는 i에서 1까지의 최소 연산 수다.

10에서 나눗셈 우선 그리디는 4회지만 DP 최적 경로는 3회다.

Lecture 10, p. 27: 최소 횟수와 경로 저장 테이블

Assignment 2: DP in Code

Assignment 2는 상태 정의와 계산 순서를 구현 선택으로 연결한다.

Problem	State	Order	Answer
Theater Seating	W(k)	k increasing	segment product
Unbounded Knapsack	dp[c]	c increasing	dp[C]
Sorting traces	current array	after required event	printed states

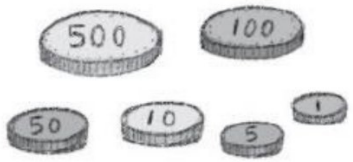
Theater Seating은 VIP가 만든 독립 구간의 경우의 수를 곱한다.
Unbounded Knapsack은 같은 물건 재사용을 위해 용량을 증가 순서로 계산한다.

Greedy Change

가장 큰 동전을 먼저 쓰는 규칙은 동전 체계의 구조가 뒷받침해야 한다.

CHANGE PROBLEM

- Greedy algorithm이 최적해가 될 수 있는 이유 :
모든 동전의 액면이 바로 아래 동전 액면의 배수이기 때문



- 이 조건이 만족하지 않으면 Greedy algorithm이 최적해 보장하지 않음
Ex) 800원을 거슬러 줘야 하는 상황



국민대학교 KOOKMIN UNIVERSITY

18

배수 사슬에서는 작은 동전 묶음을 큰 동전 하나로 교환할 수 있다.

{4,3,1}의 금액 6은 그리디 반례다.

Lecture 11, p. 18: 배수 조건과 조건이 깨지는 동전 체계

Meeting Room Allocation

가장 빨리 끝나는 회의를 고르면 남은 회의가 사용할 시간을 가장 많이 남긴다.

MEETING ROOM ALLOCATION PROBLEM

- **Greedy-choice property**
  - 종료시간이 가장 빠른 회의를 포함하는 최적해가 반드시 존재
    - 종료 시간이 빠른 순서로 **회의-1**, **회의-2**, ... 라 가정
    - **회의-1**을 포함하지 않는 최적해 T가 있다고 가정
    - T에 포함된 회의 중 종료 시간이 가장 빠른 회의가 **회의-i** 라면 T에서 **회의-i**를 빼고 **회의-1**을 넣은 해 T'는 최적해이다

- **Optimal sub-structure property**
  - 전체 회의 집합이 A이고, 종료시간이 가장 빠른 회의가 $a_1=(s_1,e_1)$이라면 최적해 S(A)는 다음과 같음

    $S(A) = S(\{a_i \in A \mid s_i \geq e_1\}) + 1$

국민대학교 KOOKMIN UNIVERSITY

25

어떤 최적해의 첫 회의를 이 선택으로 바꿔도 회의 수가 줄지 않는다.

이 교환 뒤 남은 compatible 집합도 같은 문제다.

Lecture 11, p. 25: earliest-finish 선택의 교환 논증

Final Checklist

정답보다 먼저 가정, 상태, 확정 범위, 실패 조건을 말할 수 있어야 한다.

Domain	Must explain	Common error
Analysis + Recurrence	cost unit and calls	value vs time
Sorting + Selection	fixed region or rank	all sides sorted
Dynamic Programming	state, base, order	formula memorization
Greedy	exchange proof	local rule only

수식은 조건과 함께, trace는 현재 상태와 함께 설명한다.
Greedy 규칙에는 증명 또는 반례를 붙인다.